

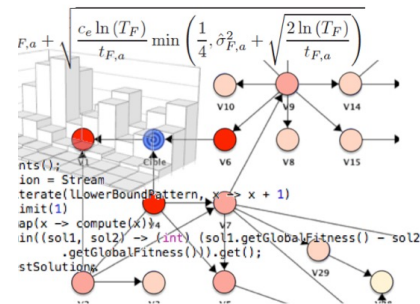
OPTIMISATION DISCRÈTE

Polytech Lyon - Filière informatique

Stéphane BONNEVAY

Polytech Lyon
Laboratoire ERIC

stephane.bonnevay@univ-lyon1.fr
me.theragoia.fr



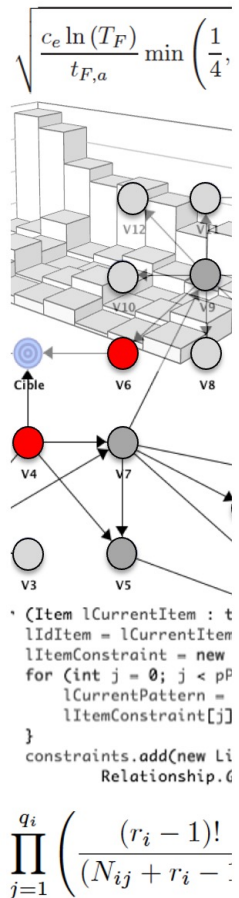
L'**objectif** est de vous présenter des méthodes génériques de résolution de problèmes d'optimisation combinatoire, problèmes dont le nombre gigantesque de solutions potentielles rend impossible la mise en œuvre d'un algorithme permettant de trouver la solution optimale en temps raisonnable.

A l'issue de ce cours, vous :

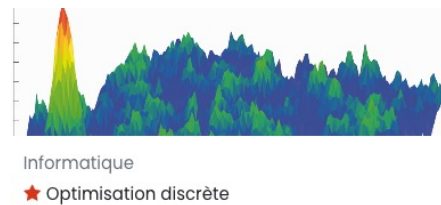
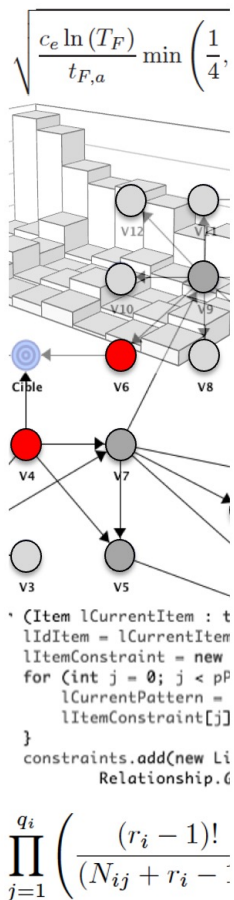
- serez capable de modéliser un problème d'optimisation, puis d'adapter ou de créer un algorithme pour résoudre ce problème
- maîtriserez les principales métaheuristiques utilisées pour la résolution de problèmes d'optimisation combinatoire
- maîtriserez la résolution de programmes linéaires

Evaluation :

- Examen : le mercredi 13 mai 2026 à 11h30
- Projet



Les cours, TD, projets et données sont téléchargeables sur **MOODLE** :



Il existe de très nombreuses **ressources pédagogiques** sur Internet et des ouvrages à la bibliothèque, comme :

- « Métaheuristiques », P. Siarry, Eyrolles, 515 pages, 2014
- « Métaheuristiques – une introduction », B. Chopard et M. Tomassini, Presses Universitaires de Perpignan, 282 pages, 2017
- « Optimisation combinatoire – théorie et algorithmes », B. Korte et J. Vygen, éditions Lavoisier, 650 pages, 2017
- ...



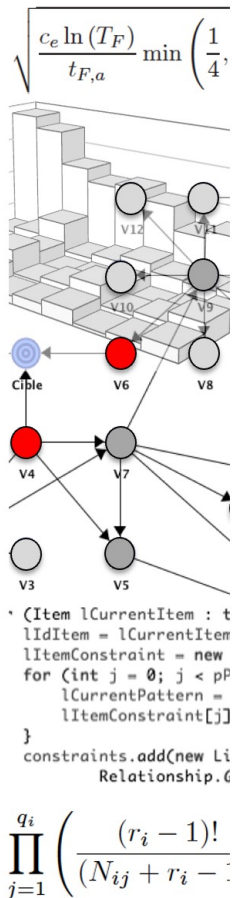
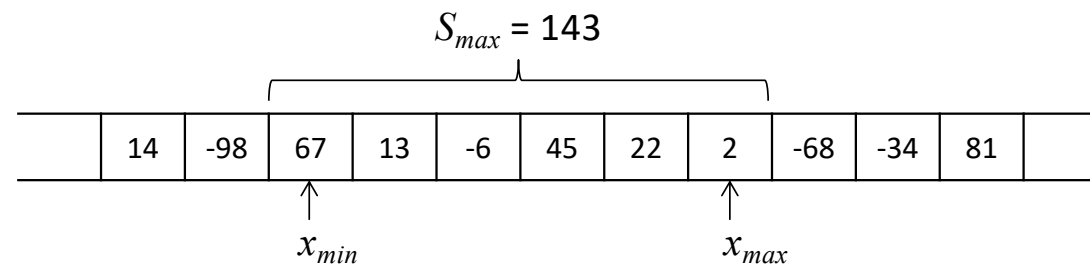
Soit un tableau $T[1, \dots, n]$ de dimension n contenant des valeurs entières prises aléatoirement dans l'intervalle discret $[-99, \dots, 99]$.

Problème :

Trouver deux entiers x_{min}, x_{max} ($1 \leq x_{min} \leq x_{max} \leq n$) tels que la quantité :

$$\sum_{i=x_{min}}^{x_{max}} T[i]$$

soit maximale (on note S_{max} la valeur maximum).



Algorithme « Trop Naïf » :

```

function TROPNAIF(T)
   $x_{min} = 1, x_{max} = 1, S_{max} = T[1]$ 
  for  $i = 1$  to  $n$  do
    for  $j = 1$  to  $n$  do
       $s = 0$ 
      for  $k = i$  to  $j$  do
         $s = s + T[k]$ 
      end for
      if  $s > S_{max}$  then
         $x_{min} = i, x_{max} = j, S_{max} = s$ 
      end if
    end for
  end for
  return  $x_{min}, x_{max}, S_{max}$ 
end function
  
```

$O(n^3)$

	14	-98	67	13	-6	45	22	2	-68	-34	81	
--	----	-----	----	----	----	----	----	---	-----	-----	----	--

Algorithme « Basic » :

```
function BASIC(T)
   $x_{min} = 1, x_{max} = 1, S_{max} = T[1]$ 
  for  $i = 1$  to  $n$  do
     $s = 0$ 
    for  $j = i$  to  $n$  do
       $s = s + T[j]$ 
      if  $s > S_{max}$  then
         $x_{min} = i, x_{max} = j, S_{max} = s$ 
      end if
    end for
  end for
  return  $x_{min}, x_{max}, S_{max}$ 
end function
```

$O(n^2)$

	14	-98	67	13	-6	45	22	2	-68	-34	81	
--	----	-----	----	----	----	----	----	---	-----	-----	----	--

Algorithme « Efficace » (algo de Kadane) :

```

function EFFICACE(T)
     $x_{min} = 1, x_{max} = 1, S_{max} = T[1]$ 
     $i_{min} = 1, s = T[1]$ 
    for  $i = 2$  to  $n$  do
        if  $s + T[i] > T[i]$  then
             $s = s + T[i]$ 
        else
             $s = T[i], i_{min} = i$ 
        end if
        if  $s > S_{max}$  then
             $x_{min} = i_{min}, x_{max} = i, S_{max} = s$ 
        end if
    end for
    return  $x_{min}, x_{max}, S_{max}$ 
end function
    
```

A chaque position i :

- soit on étend le sous-tableau
- soit on crée un nouveau sous-tableau

Meilleure Somme finissant en $i = \max($

- meilleure Somme finissant en $i-1 + T[i]$
- $T[i]$

)

$O(n)$

	14	-98	67	13	-6	45	22	2	-68	-34	81	
--	----	-----	----	----	----	----	----	---	-----	-----	----	--

Comparaison des temps de calcul (en millisecondes) :

Code en Java

```
sol.SMax = pTableau[0];
sol.xMin = 0;
sol.xMax = 0;
for (int i = 0; i < lNbElem; i++) {
    for (int j = i; j < lNbElem; j++) {
        S = 0;
        for (int k = i; k <= j; k++) {
            S += pTableau[k];
        }
        if (S > sol.SMax) {
            sol.xMin = i;
            sol.xMax = j;
            sol.SMax = S;
        }
    }
}
return sol;
```

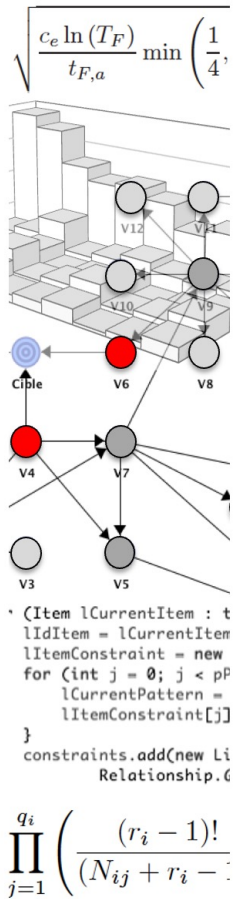
Algo « Trop Naïf »
 $O(n^3)$

```
sol.SMax = pTableau[0];
sol.xMin = 0;
sol.xMax = 0;
for (int i = 0; i < lNbElem; i++) {
    S = 0;
    for (int j = i; j < lNbElem; j++) {
        S += pTableau[j];
        if (S > sol.SMax) {
            sol.xMin = i;
            sol.xMax = j;
            sol.SMax = S;
        }
    }
}
return sol;
```

Algo « Basic »
 $O(n^2)$

```
sol.xMin = 0;
sol.xMax = 0;
sol.SMax = pTableau[0];
S = pTableau[0];
xMin = 0;
for (int i = 1; i < lNbElem; i++) {
    if ((S + pTableau[i]) > pTableau[i]) {
        S += pTableau[i];
    }
    else {
        S = pTableau[i];
        xMin = i;
    }
    if (S > sol.SMax) {
        sol.xMin = xMin;
        sol.xMax = i;
        sol.SMax = S;
    }
}
return sol;
```

Algo « Efficace »
 $O(n)$



Comparaison des temps de calcul (en millisecondes) :

	Algo « Trop Naïf »	Algo « Basic »	Algo « Efficace »
	$O(n^3)$	$O(n^2)$	$O(n)$
n = 100	3	0	0
n = 1 000	69	4	0
n = 10 000	53 000	26	1
n = 100 000	51 589 948 (> à 14h)	2 329	3

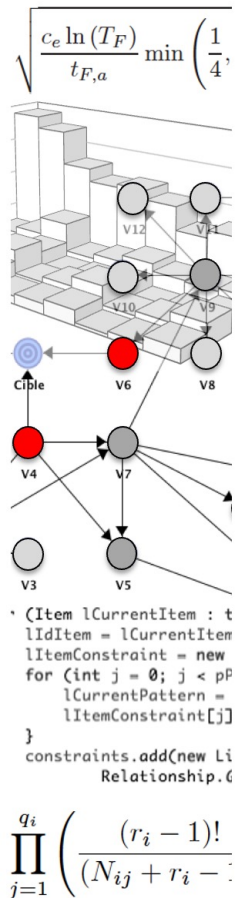
Tests réalisés en novembre 2013 sur Intel Core i7 - 2.3 GHz

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

```

        * (Item lCurrentItem : t
          lIdItem = lCurrentItem
          lItemConstraint = new
          for (int j = 0; j < pP
            lCurrentPattern =
              lItemConstraint[j]
          }
          constraints.add(new Li
            Relationship.
    
```

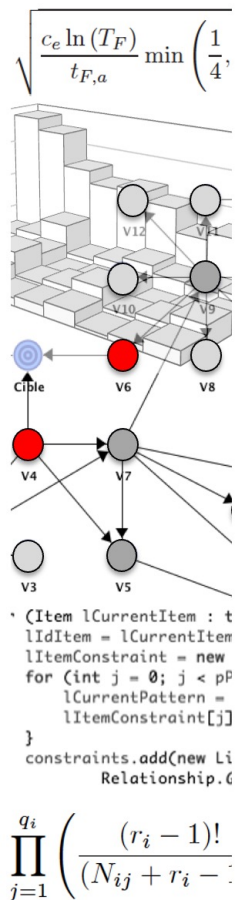
$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$



1 opération = 1 nanoseconde

n	$100n \log n$	$10n^2$	$n^{3.5}$	$n^{\log n}$	2^n	$n!$
10	3 μ s	1 μ s	3 μ s	2 μ s	1 μ s	4 ms
20	9 μ s	4 μ s	36 μ s	420 μ s	1 ms	76 années
30	15 μ s	9 μ s	148 μ s	20 ms	1 s	$8 \cdot 10^{15}$ a.
40	21 μ s	16 μ s	404 μ s	340 ms	1100 s	
50	28 μ s	25 μ s	884 μ s	4 s	13 jours	
60	35 μ s	36 μ s	2 ms	32 s	37 années	
80	50 μ s	64 μ s	5 ms	1075 s	$4 \cdot 10^7$ a.	
100	66 μ s	100 μ s	10 ms	5 heures	$4 \cdot 10^{13}$ a.	
200	153 μ s	400 μ s	113 ms	12 années		
500	448 μ s	2.5 ms	3 s	$5 \cdot 10^5$ a.		
1000	1 ms	10 ms	32 s	$3 \cdot 10^{13}$ a.		
10^4	13 ms	1 s	28 heures			
10^5	166 ms	100 s	10 années			
10^6	2 s	3 heures	3169 a.			
10^7	23 s	12 jours	10^7 a.			
10^8	266 s	3 années	$3 \cdot 10^{10}$ a.			
10^{10}	9 heures	$3 \cdot 10^4$ a.				
10^{12}	46 jours	$3 \cdot 10^8$ a.				

« Optimisation combinatoire – théorie et algorithmes. B.Korte et J.Vygen »



1. Modélisation

Modélisation et proposition de résolution de problèmes classiques

2. Métaheuristiques

métaheuristiques à base de voisinage (descente, recuit simulé, méthode Tabou, ...) et de population (algorithmes évolutionnaires, colonies de fourmis, ...)

3. Programmation linéaire

programme linéaire, méthode de résolution graphique, méthode algébrique, méthode du simplexe, dualité, programmation linéaire en nombres entiers

4. Optimisation multiobjectif

dominance de Pareto, métaheuristiques